Project Guide: Predicting Parkinson's Disease using Microbiome Data

Project Instructions

Welcome to the Project!

In this project, your team of three will work at the intersection of biology and artificial intelligence. Your goal is to train a computer program to predict whether a person has Parkinson's Disease (PD) just by looking at the bacteria living in their stomach.

If you don't know anything about biology or coding yet—don't worry! This guide will break down the science, the data, the coding, and how to write up your final report.

The Biology: What is the Microbiome?

Imagine your gut is a bustling city. The "citizens" of this city are trillions of tiny microorganisms, mostly bacteria. We call this community the **gut microbiome**.

In a healthy person, there is a good, peaceful balance of different bacterial families. But researchers have discovered that in people with Parkinson's Disease (a condition that affects the brain and nervous system), the population of this gut city changes. Some "good" bacteria families disappear, and other families multiply rapidly.

When scientists sequence DNA from stool samples, they are basically doing a census of this city. The data you will be looking at is actually just a giant spreadsheet: * Every **row** is a different patient. * Every **column** is a different type of bacteria. * The **numbers** in the cells show exactly how many of that specific bacteria were found in that patient's gut.

The Data: The Journey from Raw Counts to Hidden Patterns

When scientists run a DNA sequencer to do a "census" of the gut, it doesn't count every single bacteria perfectly. It grabs a random sample. We call the total number of bacteria successfully counted in a sample the **Total Reads**.

Because every stool sample is a little different, the sequencer might capture 40,000 bacteria for Patient 1, but only 15,000 for Patient 8.

Let's generate a realistic synthetic dataset of 100 patients (50 with Parkinson's, 50 Healthy). To show you how complex this gets, we will look at **6 different bacterial families**.

Step 1: The Raw Data

First, we will generate the raw counts and take a look at what comes straight out of the sequencing machine.

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from itables import init_notebook_mode, show

# Initialize itables for interactive spreadsheets
init_notebook_mode(all_interactive=True)
np.random.seed(42)
n_pd = 50
n_healthy = 50

# Total Reads (Let's say PD samples just happened to get sequenced deeper)
reads_pd = np.random.normal(40000, 5000, n_pd).astype(int)
reads_healthy = np.random.normal(15000, 3000, n_healthy).astype(int)
total_reads = np.concatenate([reads_pd, reads_healthy])

# Generate 6 Bacteria Columns (Underlying Proportions)
# PAIR 1 (The Hidden Rule): Bifido & Lacto (Negative correlation)
bifido_pd = np.random.uniform(0.02, 0.13, n_pd)
lacto_pd = 0.15 - bifido_pd + np.random.normal(0, 0.005, n_pd)
bifido_healthy = np.random.uniform(0.05, 0.30, n_healthy)
lacto_healthy = 0.35 - bifido_healthy + np.random.normal(0, 0.005, n_healthy)

# PAIR 2 (Distraction): Bact & Prev (No real difference between groups)
bact_pd = np.random.uniform(0.10, 0.30, n_pd)
prev_pd = 0.40 - bact_pd + np.random.normal(0, 0.01, n_pd)
bact_healthy = np.random.uniform(0.10, 0.30, n_healthy)
prev_healthy = 0.40 - bact_healthy + np.random.normal(0, 0.01, n_healthy)

# PAIR 3 (Random Noise): Esch & Kleb (Fills the rest of the gut space)
esch_pd = np.random.uniform(0.05, 0.15, n_pd)
kleb_pd = 1.0 - (bifido_pd + lacto_pd + bact_pd + prev_pd + esch_pd)
esch_healthy = np.random.uniform(0.05, 0.15, n_healthy)
kleb_healthy = 1.0 - (bifido_healthy + lacto_healthy + bact_healthy +
prev_healthy + esch_healthy)

# Convert underlying proportions to Raw Counts
df_raw = pd.DataFrame({
```

```
      "Patient_ID": [f"PD_{str(i).zfill(3)}" for i in range(1, n_pd+1)] +
[f"Healthy_{str(i).zfill(3)}" for i in range(1, n_healthy+1)],
    "Diagnosis": ["Parkinson's"] * n_pd + ["Healthy"] * n_healthy,
    "Total_Reads": total_reads,
         "Bifidobacterium": (np.concatenate([bifido_pd, bifido_healthy]) *
total_reads).astype(int),
           "Lactobacillus": (np.concatenate([lacto_pd, lacto_healthy]) *
total_reads).astype(int),
             "Bacteroides": (np.concatenate([bact_pd, bact_healthy]) *
total_reads).astype(int),
              "Prevotella": (np.concatenate([prev_pd, prev_healthy]) *
total_reads).astype(int),
             "Escherichia": (np.concatenate([esch_pd, esch_healthy]) *
total_reads).astype(int),
              "Klebsiella": (np.concatenate([kleb_pd, kleb_healthy]) *
total_reads).astype(int)
})

show(df_raw, classes="display", lengthMenu=[5, 10], scrollX=True)
```

```
<IPython.core.display.HTML object>
```

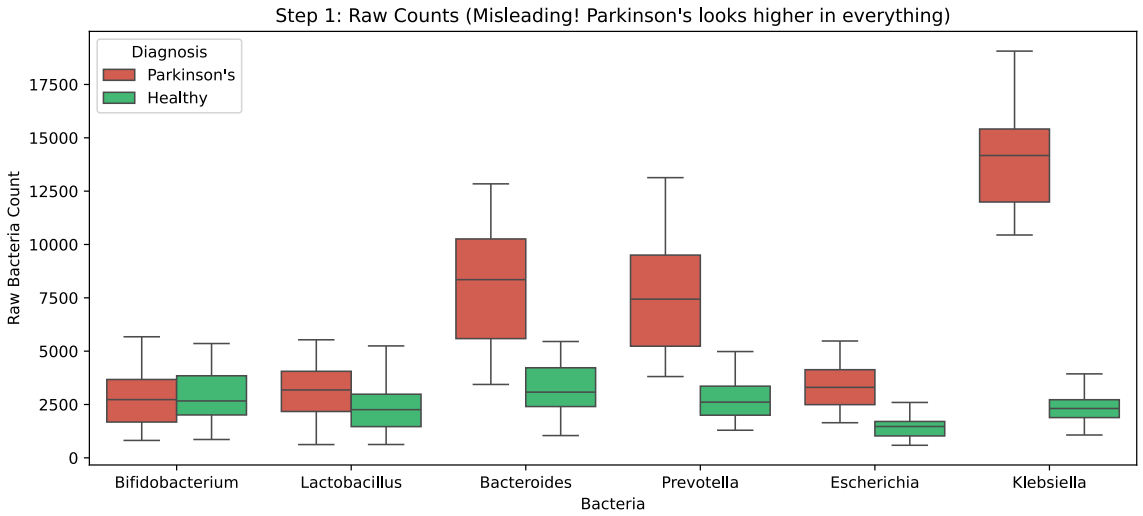
```
<IPython.core.display.HTML object>
```

If we give these raw numbers to our computer, it will get confused. Because the Parkinson's samples happened to have larger `Total_Reads` (total sequencing depth), their raw bacteria counts look really high across the board!

Let's look at a **boxplot** of this raw data. A boxplot shows the spread of the data—the box is where most patients fall, and the lines show the extremes.

```
# Reshape the data for seaborn plotting
cols = ["Bifidobacterium", "Lactobacillus", "Bacteroides", "Prevotella",
"Escherichia", "Klebsiella"]
df_raw_melted = df_raw.melt(id_vars=["Diagnosis"], value_vars=cols,
var_name="Bacteria", value_name="Raw_Count")

plt.figure(figsize=(12, 5))
sns.boxplot(data=df_raw_melted, x="Bacteria", y="Raw_Count", hue="Diagnosis",
palette=["#e74c3c", "#2ecc71"])
plt.title("Step 1: Raw Counts (Misleading! Parkinson's looks higher in
```

```
everything)")
plt.ylabel("Raw Bacteria Count")
plt.show()
```



Step 2: Converting to Proportions

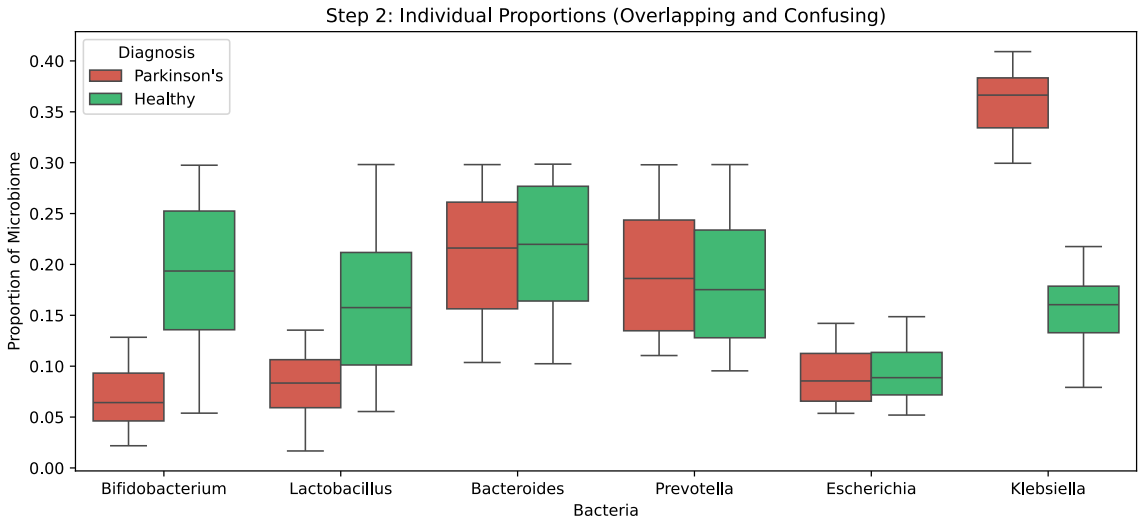
To make it a fair comparison, we must mathematically correct the data. We convert the raw counts into **proportions** (percentages) by dividing each bacteria count by the `Total_Reads` for that specific patient.

```
# Calculate Proportions
df_prop = df_raw.copy()
for col in cols:
    df_prop[col] = df_prop[col] / df_prop["Total_Reads"]

# Show the cleaned proportion data
df_view = df_prop[["Patient_ID", "Diagnosis"] + cols].round(3)
show(df_view, classes="display", lengthMenu=[5, 10], scrollX=True)
```

Now the data is mathematically sound. But if a human doctor looks at this spreadsheet, can they spot the difference between the sick and healthy patients? Let's check the boxplot of our new proportions.

```
# Reshape and plot the proportion data
df_prop_melted = df_prop.melt(id_vars=["Diagnosis"], value_vars=cols,
var_name="Bacteria", value_name="Proportion")

plt.figure(figsize=(12, 5))
```

```
sns.boxplot(data=df_prop_melted, x="Bacteria", y="Proportion", hue="Diagnosis",
palette=["#e74c3c", "#2ecc71"])
plt.title("Step 2: Individual Proportions (Overlapping and Confusing)")
plt.ylabel("Proportion of Microbiome")
plt.show()
```



Look at the boxes for *Bifidobacterium* and *Lactobacillus*. The red and green boxes completely overlap! A doctor looking at a patient with 10% *Bifidobacterium* wouldn't know which group they belong to, because healthy and sick people can both have 10%.

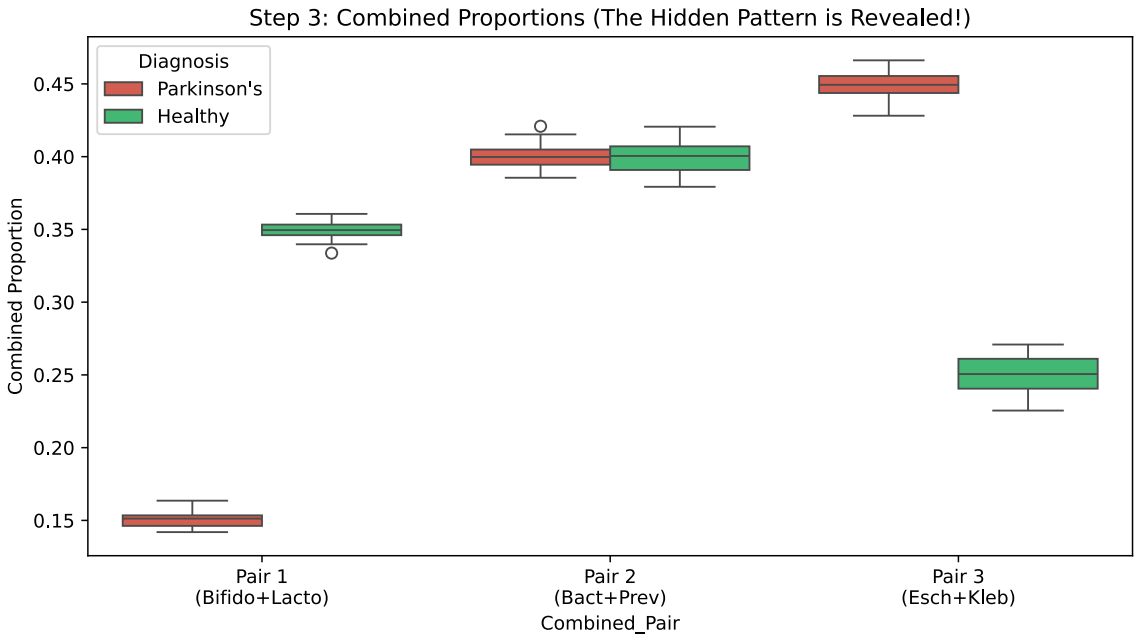
Step 3: Revealing the Hidden Pattern

Biological data is messy. Bacteria in the gut interact with each other—they compete for food and space. Sometimes, looking at one bacteria alone tells you nothing.

What happens if we combine these bacterial families into pairs? Let's add them together to create 3 new combined columns.

```
# Create 3 Combined Columns
df_prop["Pair1_Bifido_Lacto"]       =       df_prop["Bifidobacterium"]       +
df_prop["Lactobacillus"]
df_prop["Pair2_Bact_Prev"] = df_prop["Bacteroides"] + df_prop["Prevotella"]
df_prop["Pair3_Esch_Kleb"] = df_prop["Escherichia"] + df_prop["Klebsiella"]

comb_cols = ["Pair1_Bifido_Lacto", "Pair2_Bact_Prev", "Pair3_Esch_Kleb"]
df_comb_melted   =   df_prop.melt(id_vars=["Diagnosis"],   value_vars=comb_cols,
var_name="Combined_Pair", value_name="Proportion")

plt.figure(figsize=(10, 5))
```

```
sns.boxplot(data=df_comb_melted,       x="Combined_Pair",       y="Proportion",
hue="Diagnosis", palette=["#e74c3c", "#2ecc71"])
plt.title("Step 3: Combined Proportions (The Hidden Pattern is Revealed!)")
plt.ylabel("Combined Proportion")
# Make x-axis labels cleaner
plt.xticks(ticks=[0,    1,    2],    labels=["Pair    1\n(Bifido+Lacto)",    "Pair
2\n(Bact+Prev)", "Pair 3\n(Esch+Kleb)"])
plt.show()
```



The Tech: What is a Predictive Model?

If we give you a spreadsheet with hundreds of rows of patients and thousands of columns of bacteria, your human brain cannot easily spot the pattern. It's too much information.

This is where **Machine Learning** comes in. Instead of a human trying to figure out the rules, we let a computer figure it out through trial and error.

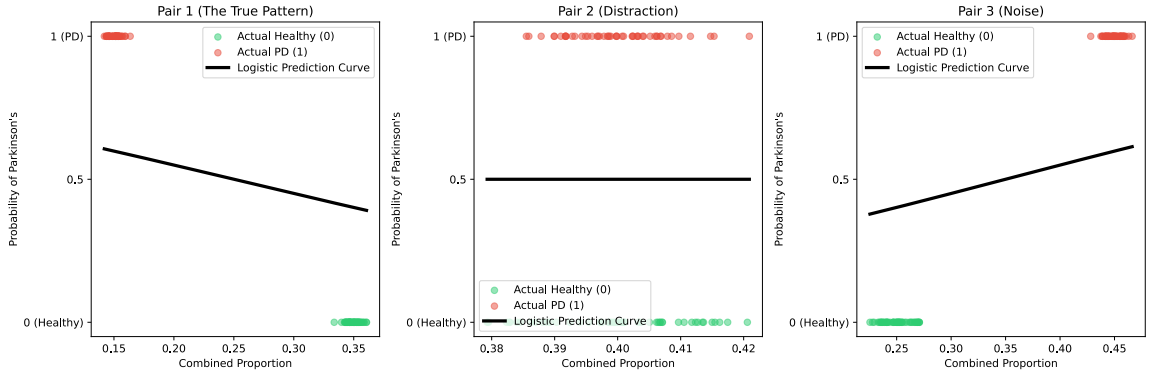
Using a Predictive Model to Describe the Separation

Before we jump into complex networks, let's look at a simpler predictive model called **Logistic Regression**.

In our previous step, we discovered that *Pair 1* (Bifidobacterium + Lactobacillus) was a great separator. How do we turn that separation into a mathematical prediction? We map the diagnosis to numbers: **Healthy = 0** and **Parkinson's = 1**. Then, we ask the computer to draw a smooth curve (an "S-curve") that connects the two.

Let's use Python to fit a Logistic Regression model to our 3 pairs and plot the results.

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.linear_model import LogisticRegression

# Convert Diagnosis to numbers: Healthy = 0, Parkinson's = 1
y = (df_prop["Diagnosis"] == "Parkinson's").astype(int)

fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(15, 5))
axes = [ax1, ax2, ax3]
pairs = ["Pair1_Bifido_Lacto", "Pair2_Bact_Prev", "Pair3_Esch_Kleb"]
titles = ["Pair 1 (The True Pattern)", "Pair 2 (Distraction)", "Pair 3 (Noise)"]

for ax, pair, title in zip(axes, pairs, titles):
    X = df_prop[[pair]].values

    # Fit the Logistic Regression model
    model = LogisticRegression()
    model.fit(X, y)

    # Create smooth line for the S-curve
    X_test = np.linspace(X.min(), X.max(), 300).reshape(-1, 1)
   y_prob = model.predict_proba(X_test)[:, 1] # Probability of being PD (Class 1)

    # Scatter plot of actual patients
     ax.scatter(X[y==0], y[y==0], color='#2ecc71', label="Actual Healthy (0)",
alpha=0.5)
       ax.scatter(X[y==1], y[y==1], color='#e74c3c', label="Actual PD (1)",
alpha=0.5)

    # Plot the regression curve
        ax.plot(X_test, y_prob, color='black', linewidth=3, label="Logistic
Prediction Curve")

    ax.set_title(title)
    ax.set_xlabel("Combined Proportion")
    ax.set_ylabel("Probability of Parkinson's")
    ax.set_yticks([0, 0.5, 1.0])
    ax.set_yticklabels(['0 (Healthy)', '0.5', '1 (PD)'])
    ax.legend()

plt.tight_layout()
plt.show()
```



Understanding the Model: Look at the black line in the first chart. It tells us exactly how to predict the disease. If a new patient walks in with a Pair 1 proportion of 0.15, the black line is at `1`, predicting they have Parkinson's. If they have a proportion of 0.35, the curve drops down to `0`, predicting they are Healthy.

For Pairs 2 and 3, the black line is flat. The model is saying, *"This data is useless, it doesn't help me predict anything."*

Why We Need Neural Networks

Logistic Regression is great, but notice that **we** had to manually combine *Bifidobacterium* and *Lactobacillus* to create Pair 1. We had to do the hard work of finding the pattern.

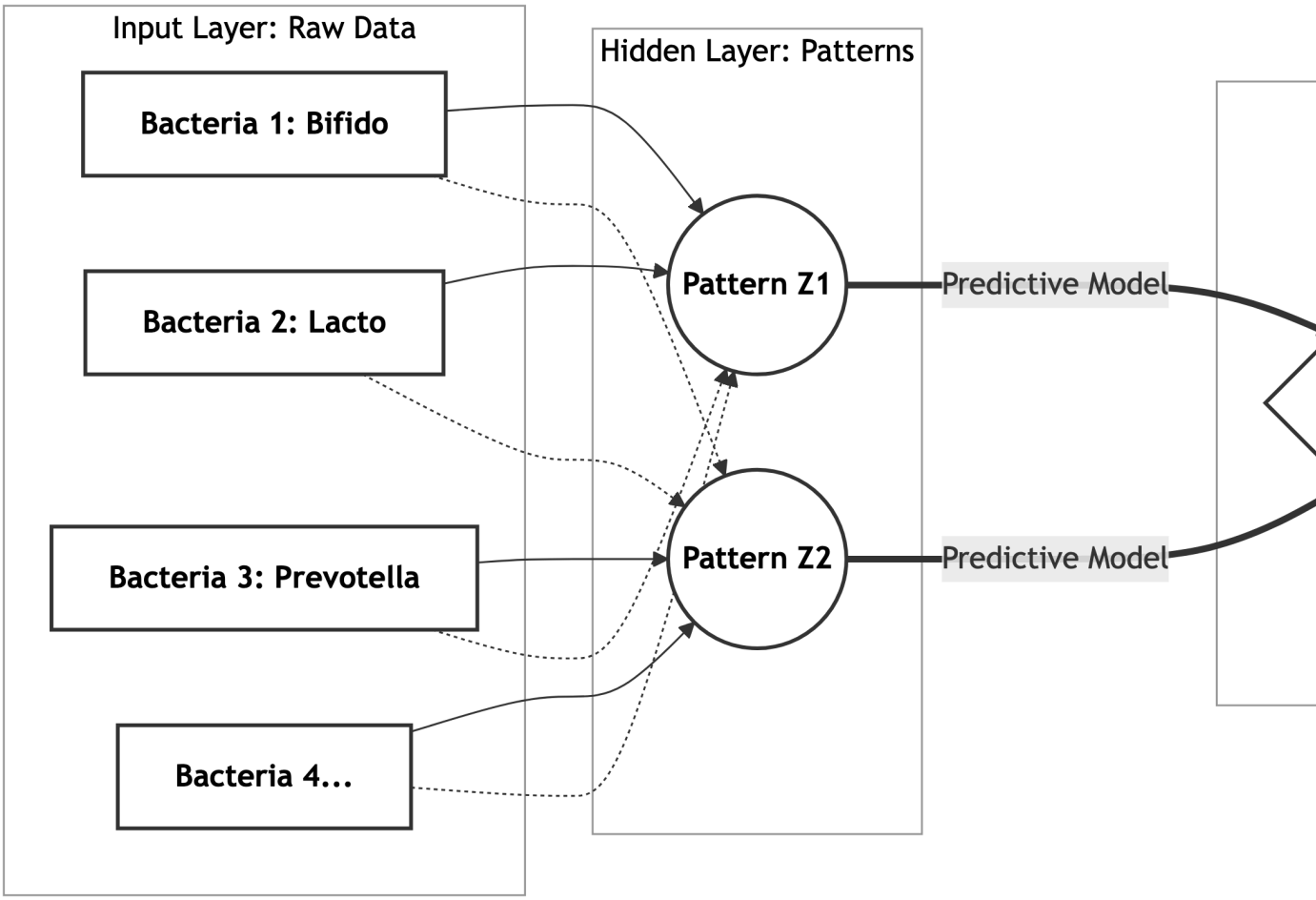
In reality, the gut has thousands of bacteria ($x_1, x_2, x_3 \ldots$). We don't know which ones to add together. We want to find some hidden, combined features (let's call them $z_1, z_2 \ldots$) that are highly predictive of the final diagnosis (y).

Instead of humans guessing the formulas, we build a **Neural Network**. We let machine learning find the mathematical functions $z_j = f_j(x)$ entirely on its own.

Think of a Neural Network like a detective agency with several floors (called **Layers**):

1. **The First Layer (Input, x):** Looks at the raw spreadsheet numbers for a patient (thousands of individual bacteria).
2. **The Middle Layers (Hidden Layers, z):** Looks for hidden patterns. It automatically tests millions of combinations to create new features (z_j). It might figure out mathematically: *"Hey, whenever Bacteria A is high and Bacteria B is low, that's a strong signal!"*
3. **The Final Layer (Output, y):** Takes those hidden signals and draws a final logistic curve to make a guess: *"Based on these patterns, the probability is 85% that this person has Parkinson's."*

Here is a visual map of what that network looks like:



How does it learn to find z? We show the network hundreds of examples where we *already know* the answer. It makes a guess. If it guesses wrong, it automatically tweaks its own internal math so it can guess better the next time. We call this **training** the model.

Collection of Microbiome Data Related To Parkinson

Because formatting raw DNA sequences is complex, we have already done the messy spreadsheet cleanup for you.

- **Core Dataset:** A preprocessed dataset is available in the `datasets` directory of this GitHub repository: PDNN GitHub Repo.
- **Data Collection Task (Student A):** One team member should take the lead on gathering additional data to test. Follow the exact steps outlined in this Google Colab Data Notebook. Additional instructions are located in the repository's "More Datasets" directory.

The Lab Bench: Computing in Google Colab

You will build your Neural Network using a tool called **TensorFlow** inside **Google Colab**. Colab is a free, cloud-based workspace that runs right in your web browser.

Setting Up Colab

1. Go to Google Colab and create a new notebook.
2. Make your code run faster by turning on the GPU: Go to `Runtime` > `Change runtime type` > select `GPU` under Hardware accelerator.

> Helpful Tutorials for Beginners
>
> If you have never coded a Neural Network before, don't panic. Run through these beginner tutorials directly in Colab before starting your main project. They walk you through the exact code you will need:
>
> - TensorFlow 2 Quickstart for Beginners
> - Basic Classification with Keras

The Writing Desk: Creating Your Report in Posit Cloud

Google Colab is fantastic for running heavy math calculations, but it is terrible for writing clean, professional reports. For that, we split our workflow: **Compute in Colab, Write in Posit Cloud.**

Posit Cloud supports **Quarto**—a system that lets you write using a "Visual Editor" (just like Google Docs or Microsoft Word) while it handles making it look like a professional science paper in the background.

Step-by-Step Writing Setup

1. Go to Posit Cloud and sign up for a free account.
2. Click **New Project** > **New RStudio Project**.
3. Go to **File** > **New File** > **Quarto Document...** Give it a title and add your names.
4. **Crucial Step:** Look at the top left corner of your document and click the **Visual** button. This switches you from code-view to a clean word processor!
5. Transfer your graphs: Save your accuracy charts in Colab as image files (e.g., `.png`), download them, and upload them to the **Files** pane in Posit Cloud. Insert them into your text using the Image icon.
6. Click the **Render** button (the blue arrow at the top) to generate your final web page or PDF.

> **Writing Tip: The "What, So What, Now What" Framework**
>
> Whenever you add a chart or result to your report, explain it using these three steps so your reader understands *why* you included it:
>
> 1. **What:** What are we looking at? *(e.g., "This graph shows our model's accuracy as it practiced over 50 rounds.")*
> 2. **So What:** Why does this matter? *(e.g., "The accuracy stops improving around round 30, meaning the model stopped learning anything new.")*
> 3. **Now What:** What did you do next? *(e.g., "Because it stopped learning, we changed the layers in our network to see if it would help...")*

Appendix: Starter Template for Your Report

Copy the text inside the box below, switch Posit Cloud to **Source** mode, paste it in, and then switch back to **Visual** mode to start writing!

```
---
title: "Predicting Parkinson's Disease from Gut Microbiome Data"
author: "Your Names Here"
format: html
---

## Introduction
<!-- Write 2-3 sentences explaining what Parkinson's Disease is and why the
bacteria in our gut might act as a clue to diagnosing it. -->

## The Dataset
<!-- Explain where the data came from. Insert a small table here showing how
many patients vs. healthy controls are in our spreadsheet. -->

## Our Neural Network Model
<!-- Insert an image of your model architecture or summary here. Explain how
many layers you chose for your "detective agency" and why. -->

## Results
<!-- Insert your accuracy charts here. Remember to use the What, So What, Now
What framework to explain them! -->

## Conclusion
<!-- Did the model successfully predict PD? What was the hardest part of building
it, and what would you try next time? -->
```